

Glitch Hunter — Tech Stack

Sep 25, 2026 · @Tanmay Singh

What this is

Glitch Hunter is a PPO reinforcement-learning agent that plays a Super Mario Bros clone and is watched live through a web dashboard; a second pipeline turns anything it catches breaking the game's own rules into an evidence bundle (screenshot, GIF, Markdown/PDF report). The table below lists every technology in the project and exactly which part of it uses that technology.

Stack by objective

Each objective built on the one before it: nothing was replaced, only added.

Objective	What it produced	Technologies it added
1 — Autonomous exploration agent	the 6M brain (46.8% level completion)	Python 3.12, Pygame, Gymnasium, PyTorch, Stable-Baselines3 PPO (CnnPolicy), NumPy, cloudpickle, Flask + Flask-SocketIO dashboard, OpenCV frame stream, HTML/CSS/JS, pytest, Git/GitHub
2 — QA / glitch-hunting agent	the 16M QA brain (84.25% coverage, 56.6% completion)	shared-memory coverage bitmap (NumPy + multiprocessing), reachability mask (.npz), QA reward and lifecycle, anchor-KL consolidation callback, 500-episode evaluation (multiprocessing Pool), SHA-256 artifact manifest, ruff, mypy, pytest-cov, GitHub Actions, uv

Objective	What it produced	Technologies it added
3 — Automated bug reporting	incidents with evidence, GIF/MD/PDF reports, replay; 6 benchmark bugs, all detected; a clean run that reaches the castle gets a "No Bugs Found" run report	incident pipeline and store, Pillow (GIF), fpdf2 (PDF), subprocess replay, zip bundles, REST API + Socket.IO events, game variants (mario_clean / mario_bugged), collision and jump-physics detectors, level-design reference file

Language, runtime and environment

Technology	Version	What it is	Where it is used	Objectives
Python	3.12 (exactly)	the programming language	the whole codebase: game wrapper, training, dashboard, reporting, tools, tests	1, 2, 3
venv (<code>venv_gpu/</code>)	—	isolated Python environment	holds every dependency; <code>run_dashboard.bat</code> and all commands use <code>venv_gpu\Scripts\python.exe</code>	1, 2, 3
pip + <code>requirements.txt</code>	—	package installer	the 2-step manual install (torch first, then <code>requirements.txt</code>)	1, 2, 3
uv + <code>uv.lock</code>	—	fast, locked installer	one-command install (<code>uv sync</code>) that encodes the torch/CUDA rules	2, 3
NVIDIA CUDA 12.6	cu126 build	GPU computing	fast PPO training on the GPU (CPU-only also works, just slower)	1, 2
Windows / Linux	Windows 11, Ubuntu (CI)	operating systems	development and dashboard on Windows; CI tests on both	1, 2, 3
Python standard library	3.12	built-in modules	<code>multiprocessing</code> (training workers, shared memory, parallel evaluation), <code>threading</code> / <code>queue</code> (dashboard game thread, report worker), <code>subprocess</code> (incident replay), <code>hashlib</code> (SHA-256), <code>json</code> , <code>zipfile</code> , <code>difflib</code> (bug-diff pin), <code>argparse</code> (tools), <code>ctypes</code> (Windows window control, power settings)	1, 2, 3

Game engine and simulation

Technology	Version	What it is	Where it is used	Objectives
Pygame	2.6.1	2D game library (on SDL 2.28)	runs the Super Mario Bros clone: drawing, sprites, collisions, the game window	1, 2, 3
Mario clone (<code>mario_clean/</code>)	vendored, pinned by SHA-256	the game itself (Level 1-1)	the untouched baseline every brain is trained and measured on	1, 2, 3
<code>mario_bugged/</code>	copy of the clone	the game with 6 declared benchmark bugs	bug-detection testing; picked from the dashboard's list	3
Gymnasium	1.3.0	standard RL environment API	<code>custom_mario_env.py</code> turns the game into an environment: 10 discrete actions, <code>step()</code> / <code>reset()</code> , info dict	1, 2, 3
Gymnasium wrappers	1.3.0	observation pipeline	frame skip 4, grayscale, resize to 84×84, stack of 4 frames <code>(wrap_observation)</code> ; <code>TimeLimit</code> in evaluation	1, 2, 3
SDL dummy video driver	—	headless rendering	training workers, evaluation, tests and incident replay run with no window	1, 2, 3
Windows <code>user32</code> via ctypes	—	native window control	<code>game_window.py</code> : centre, show, hide and minimise the dashboard's game window to the taskbar; <code>desktop.py</code> : centre the terminal and browser windows, and keep full speed on battery (power mode, process throttling)	1, 3

Reinforcement learning and machine learning

Technology	Version	What it is	Where it is used	Objectives
PyTorch	2.14.0 (+cu126)	deep-learning framework	the neural network inside the agent; runs on the GPU in training, CPU in evaluation	1, 2, 3
Stable-Baselines3 — PPO	2.9.0	RL algorithm library	trains the agent (<code>train_agent.py</code>) and loads the brain for the dashboard and evaluation	1, 2, 3
CnnPolicy	SB3	convolutional policy + value network	reads the 84×84×4 frames and chooses one of 10 actions	1, 2, 3
SubprocVecEnv / DummyVecEnv	SB3	vectorised environments	8 parallel game workers in training; a single env in the dashboard	1, 2
SB3 callbacks (custom)	project	hooks into training	checkpoints with coverage, anchor-KL consolidation, stagnation watch, value warm-up (<code>training/callbacks.py</code>)	1, 2
Anchor-KL consolidation	project	keeps a skill from being forgotten	after each update, pulls the policy back toward a frozen healthy brain if $KL > 0.08$	2
cloudpickle	3.1.2	serialisation	saving/loading checkpoints and passing objects to worker processes	1, 2, 3
TensorBoard (optional)	≥ 2.18	training-curve viewer	only if installed; training runs fine without it	1, 2
The brains	—	trained models	<code>mario_brain_checkpoint.zip</code> (6M, Objective 1) and <code>glitch_hunter_main_brain.zip</code> (16M, Objectives 2-3), frozen and hash-checked	1, 2, 3

Numerical computing, data and storage

Technology	Version	What it is	Where it is used	Objectives
NumPy	2.5.2	arrays and maths	observations, the coverage bitmap, the testable mask, jump arcs, evaluation statistics	1, 2, 3

Technology	Version	What it is	Where it is used	Objectives
<code>.npz</code> files	NumPy	compressed array files	coverage state, reachability mask, jump arcs, anchor states (<code>exploration_data/</code>), loaded with pickle disabled	2
Shared memory	stdlib <code>multiprocessing.shared_memory</code>	memory shared between processes	one global coverage map written by all 8 training workers	2
JSON	stdlib	structured records	incident records, manifests, evaluation results, <code>artifacts.json</code> , <code>INJECTED_BUGS.json</code> , <code>level1_design.json</code>	1, 2, 3
JSON Lines (<code>.jsonl</code>)	stdlib	append-only logs	coverage audit trail, reward telemetry, incident occurrences, capture failures	2, 3
SHA-256 (hashlib)	stdlib	file fingerprints	<code>artifacts.json</code> protects the brains and masks; incidents and game variants are identified by hash	2, 3
ZIP	stdlib <code>zipfile</code>	archives	model checkpoints (SB3 format), incident context frames, the "Download all" evidence bundle	1, 2, 3

Exploration and QA logic (the project's own components)

Built in-house on top of the libraries above.

Component	Location	What it does	Objectives
Game environment	<code>custom_mario_env.py</code>	drives the game one frame at a time, reports Mario's position, speed, deaths and glitches	1, 2, 3

Component	Location	What it does	Objectives
Reward wrapper	<code>agent_logic.py</code> , <code>rewards/legacy.py</code> , <code>rewards/qa.py</code>	completion reward (Objective 1); coverage + phase-gated QA reward (Objective 2)	1, 2
Spatial coverage	<code>exploration/coverage.py</code>	1-px world bitmap of everywhere Mario has been, persistent across episodes	2
Reachability / testable mask	<code>exploration/reachability.py</code>	which pixels Mario can physically reach (3 methods + real jump arcs) — the coverage denominator	2
Episode lifecycle	<code>exploration/lifecycle.py</code>	EXPLORE → COMPLETE only on evidence; safety reset on real stagnation, never on time	2
Completion evaluation	<code>evaluation/completion.py</code>	the frozen 500-episode protocol that decides HEALTHY / WARNING / REGRESSED	1, 2

Component	Location	What it does	Objectives
Game variants	<code>reporting/variants.py</code>	identifies clean vs bugged game by tree hash; every bug change must be declared and marked	3
Collision and jump-physics detectors	<code>reporting/collision_invariants.py</code>	generic rules: clipping into solids, blocked by nothing, hit without contact, impossible jump	3
Level design reference	<code>reporting/level_design.py</code> , <code>level1_design.json</code>	the drawn ground, pipes and steps the detectors compare against	3

Image and video processing

Technology	Version	What it is	Where it is used	Objectives
OpenCV (opencv-python)	5.0.0.93	image processing	resizes each game frame to 480×360 and JPEG-encodes it for the live stream; PNG-encodes the exact trigger frame of a bug	1, 3
Pillow	12.3.0	image library	builds the context GIF of each incident and prepares images for the PDF report	3
Pygame surfaces	2.6.1	raw frame capture	the full-resolution frame copied at the exact moment a detector fires	3

Bug evidence and reporting

Technology / component	Version	What it is	Where it is used	Objectives
fpdf2	2.8.8	PDF generator	the PDF bug report of every incident (<code>reporting/render.py</code>) and the run report of every clean run that reaches the castle (<code>reporting/run_report.py</code>)	3
Markdown	—	text report format	the Markdown bug report of every incident	3
Incident pipeline	<code>reporting/pipeline.py</code>	capture → store → render → replay, in the background	turns each detection into a complete incident; retries and recovers after a crash	3
Incident store	<code>reporting/store.py</code>	evidence on disk	one read-only folder per incident with a hash manifest; atomic writes; never overwritten	3
Incident schema	<code>reporting/schema.py</code>	the standard bug record	ids, timestamps, deterministic JSON, validation	3
Severity, confidence, duplicates	<code>reporting/analysis.py</code>	classification	severity per bug type, detector confidence, same-bug matching and occurrence counts	3
Replay (reproduction)	<code>reporting/reproduce.py</code> + subprocess	re-runs the episode headless	proves the bug happens again on the same frame: reproduced / not reproduced	3
Provenance	<code>reporting/provenance.py</code>	identity of brain, game and code	every report says which brain and which game produced it	3
Maintenance tool	<code>tools/incidents.py</code>	command line	list, show, verify, re-render, re-run replay	3

Web dashboard: backend

Technology / component	Version	What it is	Where it is used	Objectives
Flask	3.1.3	web framework	<code>app.py</code> : serves the page and the REST API (<code>/api/status</code> , <code>/api/incidents</code> , report and evidence downloads)	1, 2, 3

Technology / component	Version	What it is	Where it is used	Objectives
Flask-SocketIO	5.6.1	real-time messaging	pushes video frames, log lines, "Bug Found" and report-ready events to the browser	1, 2, 3
Threading mode	Flask-SocketIO	concurrency model	real OS threads (eventlet deliberately not used)	1, 2, 3
Werkzeug server	via Flask	local HTTP server	listens on 127.0.0.1:5000 only by default; no debug console	1, 2, 3
Game thread service	<code>dashboard_service.py</code>	one thread owns the game window	Start / Stop / Reset / game switch, pause on every bug, resume on Start	1, 3
Dashboard backend	<code>dashboard_backend.py</code>	env + brain + frame stream	loads the approved brain (hash-checked), plays, captures incidents	1, 2, 3
Safe file serving	<code>reporting/store.py</code> allow-list	security	only known incident ids and file names can be downloaded; path tricks get 404	3

Web dashboard: frontend

Technology	Version	What it is	Where it is used	Objectives
HTML5 (Jinja template)	—	page structure	<code>templates/index.html</code> : controls, video, Bug Tracker, Log Terminal, game selector	1, 2, 3
CSS3	—	styling	<code>static/css/style.css</code> : dark theme and responsive layout	1, 2, 3
Vanilla JavaScript	ES2017+	page behaviour, no framework	<code>static/js/main.js</code> : buttons, live video, Bug Tracker cards, banner, reset, game switch	1, 2, 3
Socket.IO JS client	vendored (<code>static/vendor/</code>)	browser side of the live connection	receives frames and events; kept locally so the dashboard works offline	1, 2, 3
Fetch API	browser	REST calls	loads status and the session's incidents	3

Testing, code quality and CI/CD

Technology	Version	What it is	Where it is used	Objectives
pytest	9.1.1	test framework	~730 automated tests in <code>tests/</code> (fast ones by default, <code>-m slow</code> for brain and end-to-end)	1, 2, 3
pytest-cov	7.1.0	code coverage	the full gate requires at least 90% (last run 90.64%)	2, 3
Ruff	0.16.6	linter	style, bugs and security (bandit) rules on every run	2, 3

Technology	Version	What it is	Where it is used	Objectives
mypy	2.3.1	static type checker	strict mode, checked for both Windows and Linux	2, 3
pre-commit	4.6.2	git hooks	runs checks before each commit	2, 3
Quality gate	<code>tools/check.py</code>	one command for all checks	lint → types → tests → artifact hashes (<code>--full</code> adds slow tests + coverage floor)	2, 3
Artifact verification	<code>tools/verify_artifacts.py</code>	integrity check	re-hashes the protected brains, masks and baseline against <code>artifacts.json</code>	2, 3
End-to-end validation	<code>tools/validate_incident_pipeline.py</code>	36-check pipeline proof	runs the real dashboard stack headless and checks every reporting stage	3
Git + GitHub	—	version control and hosting	source history and the remote repository	1, 2, 3
GitHub Actions	<code>.github/workflows/ci.yml</code>	continuous integration	lint, types and tests on Ubuntu and Windows on every push	2, 3

Tooling, scripts and documentation

Tool / format	Where it is used	Objectives
<code>train_agent.py</code>	training, with safe resume (<code>--resume-from</code> , <code>--dry-run-resume</code>)	1, 2
<code>tools/evaluate_completion.py</code>	500-episode completion test (HEALTHY / WARNING / REGRESSED)	1, 2

Tool / format	Where it is used	Objectives
<code>tools/build_reachability.py</code> , <code>collect_jump_arcs.py</code> , <code>bootstrap_coverage.py</code> , <code>migrate_coverage.py</code> , <code>build_anchor_set.py</code> , <code>calibrate_*.py</code>	one-time setup and measurement for the QA campaign	2
<code>tools/build_level_design.py</code>	writes the level-design reference from the clean game	3
<code>run_dashboard.bat</code>	double-click to start the dashboard and open the browser	3
Markdown docs	<code>README.md</code> , <code>docs/OBJECTIVE1-3.md</code> , <code>docs/ARCHITECTURE.md</code> , <code>docs/PERFORMANCE.md</code> , <code>SECURITY.md</code> , <code>CONTRIBUTING.md</code>	1, 2, 3
Mermaid diagrams	the architecture diagram in <code>docs/ARCHITECTURE.md</code>	2, 3

Notes

- **torch is pinned to 2.14.0** (CUDA 12.6 build): it has no published security advisories, and it meets Stable-Baselines3's own torch requirement, so SB3 installs normally. It replaced 2.5.1 (CUDA 12.1) on 2026-09-25; both brains play move-for-move the same on it. Install torch first (so Windows gets the GPU build), then `requirements.txt` — or simply `uv sync --active`.
- **eventlet must never be installed.** Flask-SocketIO picks its async mode from what is installed; the dashboard runs in plain threading mode on purpose.
- **Headless everywhere except the dashboard:** training, evaluation, tests and incident replays use the SDL dummy video driver; only the dashboard opens a real game window.
- **Artifacts are protected by hash, not by git:** the brains, masks and coverage files are large and git-ignored; `artifacts.json` (tracked) holds their SHA-256 values.